RabbitMQ : corrigé du TP étudiant

Réponses, observations et compléments de code

Support associé : TP-RabbitMQ-etudiant.pdf, version en 12 pages. Les repères Q1 à Q7 et les TODO H, T1 et T2 correspondent exactement à ce support et à son kit. Une distribution précise des tâches entre workers n'est pas un critère de réussite.

Partie 1 : manipuler un message · Q1

Action dans demo.test	Résultat attendu après actualisation
Publication de Salut	Un message disponible : Ready=1.
Nack message requeue true	Le contenu est affiché puis remis en queue : il est disponible de nouveau.
Nouvelle lecture avec remise	Le même contenu peut être relu.
Automatic ack	Le message est acquitté et retiré de la queue.
Lecture suivante	L'interface indique Queue is empty.

Q1. Lire un message ne prouve pas que le travail demandé a été terminé. L'acquittement indique à RabbitMQ qu'il peut considérer cette livraison comme traitée et retirer le message. Dans l'interface, Automatic ack acquitte la lecture sans représenter un traitement métier.

`Ready` compte les messages disponibles pour livraison. `Unacked` compte les messages livrés pour lesquels le broker attend encore un acquittement. Les lectures de l'interface sont brèves : elles ne permettent pas nécessairement d'observer un état Unacked durable. Les métriques peuvent rester anciennes quelques secondes ; une réponse de lecture et un compteur peuvent donc sembler se contredire momentanément.

Partie 2 : premier envoi · Q2 et Q3

Q2. Le producer peut publier avant le démarrage du consumer : la queue hello existe déjà, car le producer la déclare. Elle conserve le message jusqu'à sa consommation. Le producer n'attend pas que le consumer réalise le traitement.

Premier envoi sans consumer : Ready passe à 1. Le consumer démarre, affiche le contenu et l'acquitte ; Ready et Unacked finissent à 0. Deux publications supplémentaires produisent deux nouveaux affichages lors d'un essai normal, sans panne ni autre consumer concurrent.

Q3. Chaque programme déclare la queue pour que l'ordre de démarrage soit libre. Une déclaration est idempotente si les paramètres sont identiques. Une nouvelle déclaration incompatible, par exemple une autre durabilité, entraîne une erreur de canal PRECONDITION_FAILED. Cette déclaration ne crée pas une deuxième queue portant le même nom.

Via l'exchange par défaut, une publication dont aucune queue ne correspond à la routing key n'est pas conservée dans ce TP. Aucun mécanisme de retour obligatoire ni alternate exchange n'est configuré ici.

Partie 3 : work queue et panne

Q4 et Q5 · Un traitement successif par worker

Deux workers sur une queue · Q4

Les workers A et B consomment la même queue tasks.email : ils **se partagent** les tâches. Une livraison donnée est attribuée à un worker. Une redélivrance peut cependant faire traiter à nouveau une tâche après une interruption ; il ne faut donc pas conclure à une exécution unique garantie.

Observation attendue	Interprétation
A et B traitent des numéros différents pendant l'essai normal	Ils partagent le travail d'une même queue.
Au plus un message non acquitté par worker avec prefetch=1	Le broker limite les livraisons en attente d'acquittement pour chaque consumer.
Ready peut rester positif pendant le traitement	Des tâches attendent qu'un worker puisse les recevoir.
Ready=0 et Unacked=0 à la fin	Aucune tâche ne reste disponible ou en cours non acquittée.

La liste précise des numéros n'est pas imposée. Le démarrage des consumers et les durées de traitement influencent la distribution. Avec deux workers connectés et ce réglage, on peut voir jusqu'à deux messages Unacked au total ; le rafraîchissement des métriques peut masquer cet état.

Le prefetch et le moment de l'ack · Q5

Q5. Un prefetch de 1 autorise au plus une livraison non acquittée par worker dans cette expérience. Après l'ack, le worker peut recevoir la suivante. Cela limite la réservation anticipée de tâches ; cela ne garantit ni des durées de travail identiques ni un ordre global de fin.

Dans les workers Python et Node fournis, l'ack est envoyé après l'attente qui simule le travail. En Java, le container Spring utilise AcknowledgeMode.AUTO : il acquitte après le retour normal du listener. Ce mode Spring n'est pas l'auto-ack immédiat du protocole AMQP.

Avec un acquittement immédiat, une panne survenant ensuite ne permet pas au broker de savoir que le traitement est incomplet : la tâche peut être perdue du point de vue métier.

L'expérience d'interruption

1. B est arrêté et une tâche de 30 secondes est livrée à A. Elle est en cours de traitement, non acquittée.

2. A est interrompu avant la fin. Après détection de la fermeture, sa livraison non acquittée est remise à disposition. Sans autre consumer actif, on peut observer Ready=1 et Unacked=0 après actualisation.

3. B redémarre, reçoit la tâche avec l'indication de redélivrance, la traite puis l'acquitte. Les compteurs finissent à 0.

Si B était resté actif, il pourrait recevoir la tâche très vite : l'état Ready=1 pourrait être invisible. Pour Java, un arrêt gracieux peut laisser finir le listener ; la fiche étudiant demande donc une terminaison brutale du seul processus visé.

Partie 4 : routage topic

Q6 et Q7 · Une queue par service

Lot initial : prédictions et observations attendues

Événement	Stock : commande.*	Notifications : commande.creee	Audit : #
commande.creee	Reçu	Reçu	Reçu
commande.annulee	Reçu	Non reçu	Reçu
paiement.refuse	Non reçu	Non reçu	Reçu

Le premier lot produit deux réceptions chez Stock, une chez Notifications et trois chez Audit, si les queues sont vides au départ et qu'aucune liaison supplémentaire n'est présente.

Chaque service possède sa propre queue. Un même événement peut donc être routé vers plusieurs queues liées, puis consommé indépendamment. Deux instances du service Stock consommant stock.queue partageraient les messages de cette queue ; elles n'en recevraient pas chacune systématiquement une copie.

L'existence des queues et des bindings · Q6

Q6. L'exchange dirige les publications, mais ne les stocke pas en attendant la création de futurs abonnés. Vérifier les queues et leurs bindings avant la publication permet d'éviter une publication sans destination correspondante.

Un consumer actif n'est pas indispensable pour recevoir une publication **dans une queue** : si la queue et son binding existent, elle peut conserver les messages en attendant le consumer. Démarrer les consumers avant le publisher sert ici à installer et vérifier cette topologie.

Pour tester une publication avant tout binding, utiliser un nouvel exchange topic dédié, sans liaison : y publier un message, puis créer une queue et la lier avec #. La queue ne contient pas l'ancienne publication. Il faut employer de nouveaux noms pour que des bindings conservés d'un précédent essai ne faussent pas l'expérience. Dans ce scénario sans mécanisme de retour ni destination alternative, le message non routé est perdu.

Livraison et l'événement urgent · Q7

Après ajout de livraison.queue liée avec commande.#, **commande.creee.urgent est reçu par Audit et Livraison**, pas par Stock ni Notifications.

Q7. commande.* accepte exactement un segment après commande. Le motif ne correspond donc pas à deux segments supplémentaires, creee et urgent. commande.# accepte zéro ou plusieurs segments après commande ; # seul correspond à l'ensemble des clés de cet exemple. La clé exacte commande.creee ne correspond pas à commande.creee.urgent.

Après arrêt de tous les consumers, publier commande.creee laisse un message en attente dans chacune des quatre queues liées de cet essai. Au redémarrage, les consumers les reçoivent. Les queues doivent avoir été vidées avant ce test pour interpréter les nombres.

Mini rendu : réponses possibles

Évaluer la compréhension, pas une formulation unique

1. Deux architectures, deux usages

Réponse possible. Dans la partie 3, plusieurs workers prennent des tâches dans tasks.email pour partager le travail. Dans la partie 4, Stock, Notifications et Audit ont chacun une queue : les événements peuvent être envoyés à plusieurs services selon les bindings. On augmente le nombre de workers d'une même queue pour partager ses tâches ; on crée des queues distinctes pour des besoins de consommation indépendants.

Ce qu'on cherche : distinguer concurrence sur une queue et diffusion vers plusieurs queues. « Tous les consumers reçoivent toujours chaque message » est une généralisation incorrecte.

2. Une panne pendant le travail

Réponse possible. La tâche a été livrée mais pas acquittée. Quand la connexion du worker s'est fermée, le broker l'a rendue disponible. Le nouveau worker l'a reçue et traitée. Le même traitement peut donc être tenté de nouveau.

Même si l'envoi réel d'un e-mail avait déjà réussi, une panne juste avant l'ack pourrait provoquer une autre tentative. Une protection métier contre les doublons peut utiliser un identifiant de tâche et une trace fiable des opérations déjà réalisées. L'organisation exacte dépend du stockage et des effets métier ; un simple ack tardif ne rend pas le traitement automatiquement idempotent.

Ce qu'on cherche : relier fermeture de connexion, absence d'ack et redélivrance ; reconnaître le risque de doublon sans affirmer une exécution exactement une fois.

3. Une règle de routage

Réponse possible. Audit reçoit commande.creee.urgent grâce au binding # ; Livraison grâce à commande.#. Stock utilise commande.*, qui n'accepte qu'un segment après commande. Notifications utilise une correspondance exacte.

Ce qu'on cherche : raisonner sur les segments de la clé plutôt que sur les noms des services.

4. Une difficulté et une vérification

Plusieurs réponses conviennent : compteur pas encore actualisé ; ancien consumer encore actif ; oubli de guillemets ; programme Java non recompilé ; ancien binding conservé ; publication effectuée avant le démarrage des consumers qui déclarent les queues.

Ce qu'on cherche : une observation précise et une vérification cohérente. Une prédiction initiale fausse, corrigée grâce à l'expérience, est un signe d'apprentissage et ne doit pas être pénalisée en elle-même.

Critères de réussite

- L'étudiant explique où le message attend avant sa consommation.
- Il distingue message disponible et livraison non acquittée.
- Il situe l'ack après le traitement et explique un scénario de doublon.
- Il distingue plusieurs workers d'une queue et plusieurs queues abonnées.
- Il justifie les destinataires d'une routing key à partir des motifs.

Compléter les TODO : Python

Remplacer les instructions NotImplementedError correspondantes

La connexion, les déclarations, la consommation et les acquittements sont fournis dans le kit. Les extraits ci-dessous remplacent uniquement les zones demandées ; conserver le reste des fichiers et leur indentation.

TODO H · hello-send.py

Après la déclaration de hello, remplacer l'erreur volontaire par :

```
ch.basic_publish(
    exchange='',
    routing_key='hello',
    body='Bonjour RabbitMQ !'.encode()
)
```

Le nom vide désigne l'exchange par défaut. La routing key indique ici la queue cible. Ce premier message sert à observer envoi et consommation ; le sender de tâches fourni introduit séparément la persistance et les confirmations.

TODO T1 · topic-consumer.py

À l'intérieur de la boucle `for key in keys`, remplacer l'erreur volontaire par :

```
ch.queue_bind(
    queue=queue,
    exchange='events',
    routing_key=key
)
```

Dans cet appel de binding, le paramètre nommé routing_key porte la binding key. La variable queue vaut le nom du service suivi de .queue. Chaque clé passée en argument crée une liaison supplémentaire ; les anciennes liaisons ne sont pas supprimées par un nouveau lancement.

TODO T2 · topic-publish.py

À l'intérieur de la boucle de publication, la variable body est déjà un objet bytes. Remplacer l'erreur volontaire par :

```
ch.basic_publish(
    exchange='events',
    routing_key=key,
    body=body
)
```

Vérifier le résultat

Envoyer le premier Hello World avant de démarrer son consumer, puis refaire deux envois avec le consumer actif. Pour les topics, démarrer les consumers Stock, Notifications et Audit et attendre leur message d'attente avant d'envoyer le lot. Comparer les affichages au tableau de la partie 4.

Le callback du worker est synchrone : il attend dans traiter(body), puis envoie basic_ack. La fermeture de sa connexion pendant le traitement laisse cette livraison non acquittée et permet sa remise à disposition.

Compléter les TODO : Node.js

Remplacer les instructions throw new Error correspondantes

Conserver les appels waitForConfirms() et la fermeture de connexion fournis dans les publishers. Les channels créés par ouvrir() sont des ConfirmChannel.

TODO H · hello-send.js

La variable body est déjà un Buffer. Remplacer l'erreur volontaire par :

```
ch.sendToQueue('hello', body);
```

Conserver ensuite `await ch.waitForConfirms()`. sendToQueue publie via l'exchange par défaut. La valeur booléenne renvoyée par la méthode relève du contrôle du tampon d'écriture, pas d'un acquittement du broker.

TODO T1 · topic-consumer.js

À l'intérieur de `for (const key of keys)`, remplacer l'erreur volontaire par :

```
await ch.bindQueue(queue, 'events', key);
```

La déclaration de l'exchange et de la queue a déjà été faite. La liaison doit être installée avant la première publication que l'on souhaite recevoir dans cette queue.

TODO T2 · topic-publish.js

À l'intérieur de la boucle de publication, remplacer l'erreur volontaire par :

```
ch.publish('events', key, body);
```

Conserver l'attente de confirmation après la boucle, avant la fermeture. Cette confirmation concerne la prise en charge de la publication par le broker ; elle ne prouve pas que tous les consumers ont fini leur traitement, ni qu'un message non obligatoire a trouvé une queue correspondante.

Pourquoi le worker fourni traite successivement

Le callback ajoute le traitement à une chaîne de promesses suite. Un traitement commence après la fin du précédent ; l'ack est envoyé après `await traiter(msg)`. Ainsi, même avec prefetch=6 dans le bonus, le worker ne simule pas six travaux en parallèle.

Déclarer simplement le callback async ne ferait pas attendre RabbitMQ entre les callbacks. Plusieurs setTimeout indépendants pourraient finir simultanément et brouiller la comparaison avec Python et Java. La sérialisation fournie est volontaire pour cette expérience pédagogique.

Vérifier le résultat

Utiliser les commandes de la fiche Node du TP étudiant. Deux tâches d'une seconde sur un seul worker doivent prendre environ deux secondes au total, hors démarrage : c'est un repère pour vérifier que la simulation reste successive. La répartition entre deux workers se constate ensuite dans leurs terminaux.

Compléter les TODO : Java / Spring AMQP

Remplacer le corps des trois méthodes dans TpApplication.java

Les imports du kit incluent déjà les classes utilisées. Recompiler avec `mvn -q -DskipTests package` après modification, puis redémarrer les processus concernés avec le nouveau jar.

TODO H · publierHello()

```
private void publierHello() {
    rt.convertAndSend("", "hello", "Bonjour RabbitMQ !");
}
```

Cette surcharge précise l'exchange vide, la routing key et le contenu. La queue hello est déclarée dans le rôle avant cet appel.

TODO T1 · lierTopic(...)

```
private void lierTopic(String queue, String key) {
    Binding binding = new Binding(
        queue,
        Binding.DestinationType.QUEUE,
        "events",
        key,
        null
    );
    admin.declareBinding(binding);
}
```

Construire un objet Binding ne suffit pas : sa déclaration par RabbitAdmin installe la liaison sur le broker. Un BindingBuilder équivalent est également une réponse correcte.

TODO T2 · publierTopic(...)

```
private void publierTopic(String key, String body) {
    rt.convertAndSend("events", key, body);
}
```

Le rôle topic-publish appelle la méthode pour chaque clé de tp.events ou, en l'absence de cette option, pour le lot par défaut. Les rôles publisher et consumer peuvent être lancés indépendamment dans des processus distincts.

Acquittement et prefetch

Le listener fourni est configuré avec une seule instance consommatrice par processus et un prefetch explicite. Son mode AUTO laisse Spring acquitter après le retour normal du callback. Aucun basicAck manuel n'est à ajouter à cette version.

Le sender de tâches utilise des messages persistants et attend des confirmations dans un canal dédié via RabbitTemplate.invoke. Pour l'expérience de panne, terminer brutalement le PID affiché par le worker évite de confondre panne et arrêt gracieux.

Bonus : interpréter les expériences

Limites et points de vigilance

A. Prefetch=6 comparé à prefetch=1

Avec 6, un worker peut recevoir jusqu'à six livraisons non acquittées. Comme il traite les tâches successivement, certaines peuvent attendre dans son client tandis que l'autre worker est déjà libre. Avec 1, le broker attend l'ack avant de lui attribuer une nouvelle livraison.

Un exemple possible avec le lot alternant cinq secondes et une seconde : avec une réservation anticipée importante, A traite 1,3,5 en environ 15 secondes et B traite 2,4,6 en environ 3 secondes. Avec 1, on peut observer A traitant 1,4,6 en environ 7 secondes et B traitant 2,3,5 en environ 11 secondes. Ces listes sont des exemples, pas des garanties.

Les deux workers doivent être connectés avant la publication, les tâches précédentes terminées et la queue vide. Une différence de démarrage, d'ordonnancement ou de traitement peut changer les résultats. Le prefetch limite des livraisons non acquittées ; il n'est pas, à lui seul, une définition de la concurrence métier.

B. Durabilité et redémarrage

Avec le sender fourni, les trois tâches du lot durable sont publiées comme persistantes dans une queue durable ; le sender attend leur confirmation. Tous les workers étant arrêtés, on attend trois messages avant puis après docker restart, avant leur consommation.

Une queue durable permet de restaurer sa définition après redémarrage ; la persistance du message porte sur sa conservation. Une confirmation de publication renseigne le publisher sur la prise en charge par le broker. Elle ne signifie pas que le consumer a fini son travail.

Cet essai de redémarrage contrôlé ne prouve pas l'absence de perte dans toutes les pannes. Il ne teste ni la disparition du stockage ni une architecture répliquée. Le compose du kit n'installe pas de volume nommé : supprimer et recréer le conteneur ne doit pas être confondu avec un simple redémarrage et peut donner un autre stockage.

C. DLQ, rejet et expiration

Avec le DLX et le binding configurés, Reject requeue false envoie le message de demo.work vers demo.dlx, puis vers demo.dlq grâce à la clé dead. Les en-têtes x-death indiquent notamment le motif rejected et la queue d'origine. Avec le TTL, le motif observé est expired.

Une remise en queue demande une nouvelle livraison depuis la queue d'origine ; elle ne déclenche pas à elle seule le chemin de rejet vers le DLX. Une fermeture de consumer avec un message non acquitté entraîne une remise à disposition ; ce n'est pas une stratégie automatique de DLQ après un nombre fixe d'échecs pour les queues Classic du TP.

Pour un message empoisonné, il faut définir une politique explicite de tentatives puis de rejet sans remise en queue, ou une architecture de retry adaptée. Sans DLX configuré, un rejet sans remise peut entraîner la suppression du message. Les réglages avancés dépendent notamment du type de queue ; ils ne sont pas évalués dans ce TP.

Vérification et usage du corrigé

Repères pour l'enseignant

Valider sans imposer une chronologie fragile

Situation	Vérification utile
Le compteur paraît incohérent	Attendre le calcul des métriques et le rafraîchissement ; comparer à une lecture ou au terminal.
Une tâche n'arrive pas au worker attendu	Vérifier les consumers actifs et la réservation de messages ; ne pas imposer A/B à l'avance.
Le test de panne ne montre pas Ready=1	Vérifier que l'autre worker est arrêté et que la connexion du worker interrompu a été fermée.
Des événements inattendus arrivent	Vérifier les anciens messages et les bindings conservés.
Le code Java ne change pas	Recompiler le jar et redémarrer les processus concernés.

Ce qui a été vérifié pour préparer les supports

Le parcours Python initial a été exécuté contre RabbitMQ 4.4.0 : lecture et acquittement, partage des tâches, interruption et reprise, redémarrage, routage topic, publication sans consumers, rejet vers DLQ et expiration TTL.

Les consumers Hello World, les workers successifs avec prefetch=1 et 6 et les senders de tâches fournis dans le kit étudiant ont ensuite été vérifiés dans les trois langages. Le projet Java compile. Les solutions H, T1 et T2 ont également été exécutées dans les trois langages : envoi Hello World, bindings, routage du lot et de l'événement urgent, puis publication avec les consumers arrêtés.

Ces vérifications techniques ne mesurent pas la durée d'apprentissage. Prévoir les installations avant la séance ; garder les bonus facultatifs. Une séance pilote avec un binôme débutant reste utile pour ajuster le temps consacré aux étapes.

Ressources officielles

• Tutoriels : https://www.rabbitmq.com/tutorials

• Acquittements et confirmations : https://www.rabbitmq.com/docs/confirms

• Routage topic : https://www.rabbitmq.com/tutorials/tutorial-five-python

• Dead letter exchanges : https://www.rabbitmq.com/docs/dlx

• API amqplib : https://amqp-node.github.io/amqplib/channel_api.html

• API Pika : https://pika.readthedocs.io/en/stable/modules/channel.html

• Spring AMQP : https://docs.spring.io/spring-amqp/reference/

Organisation du dépôt. La branche main contient le support étudiant et le kit à compléter. La branche correction contient en plus ce corrigé. Le support étudiant demeure la référence pour les consignes, les durées et les commandes de lancement.